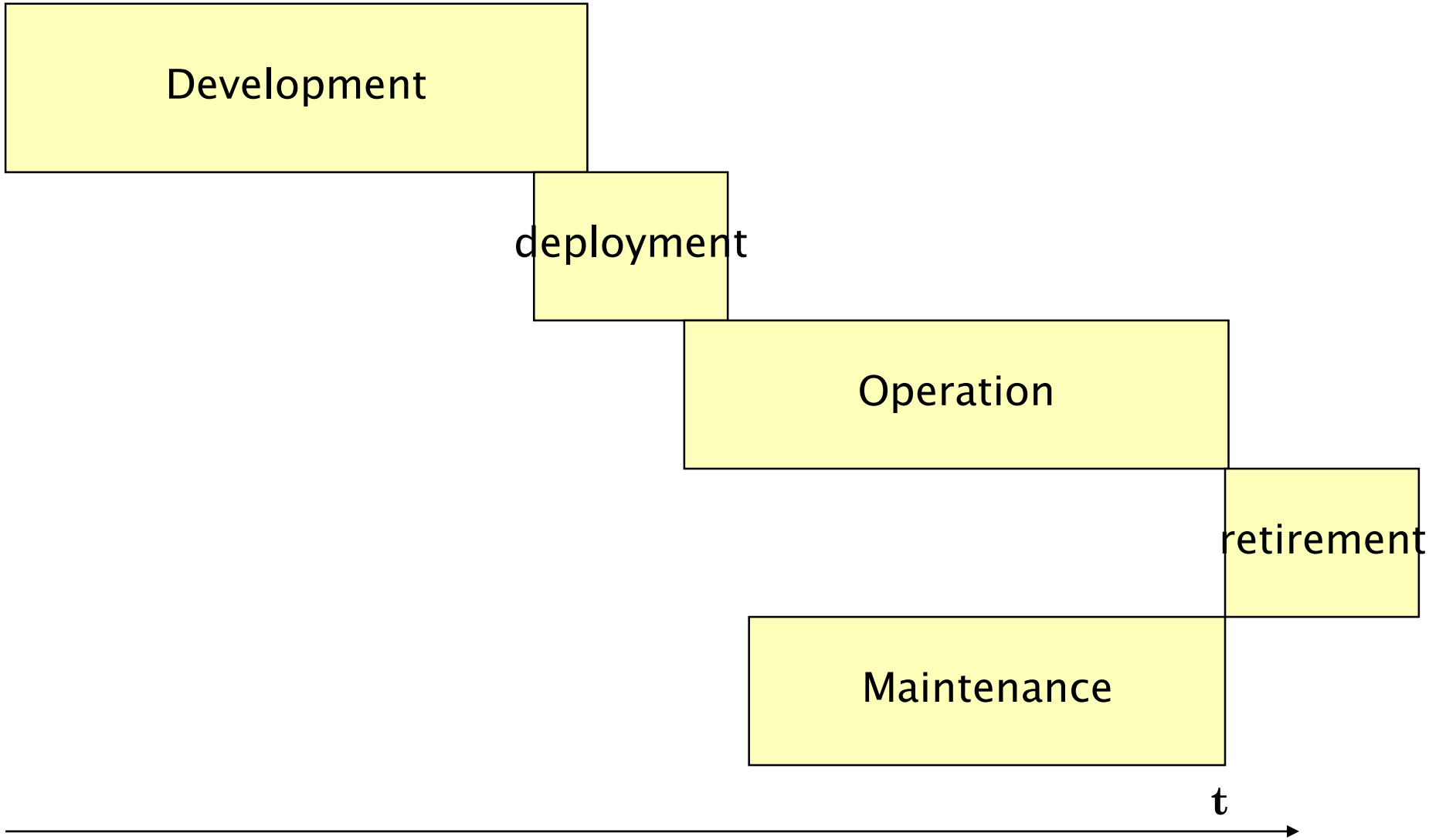
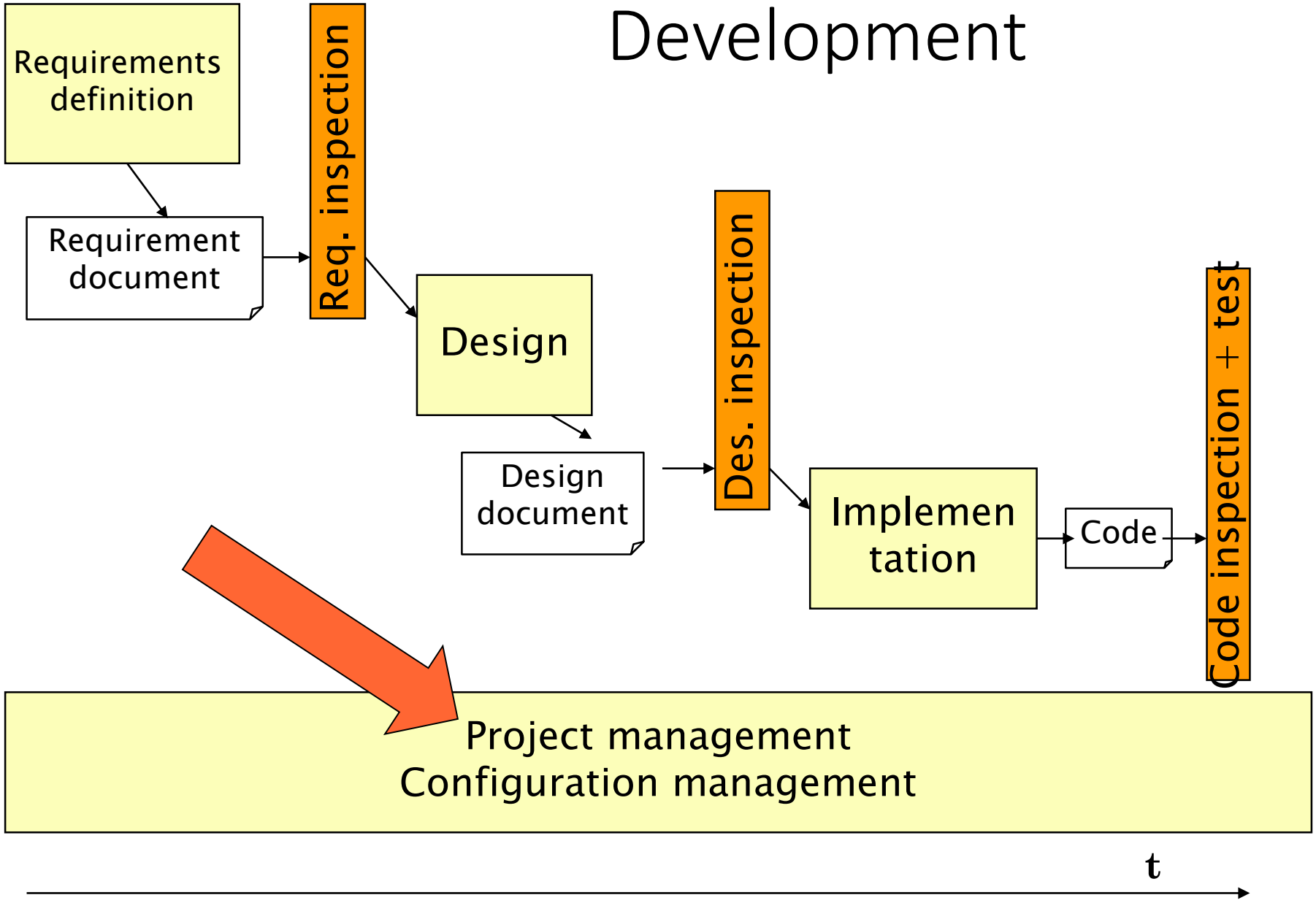
Project Management

Main Phases



Development



Project management

- How much?
  - Upfront: Estimation
  - During and after: tracking
- When ready?
  - Upfront: Estimation, scheduling
  - During and after: tracking
- Who does what?
  - Team organization, scheduling, work allocation

Resource

- Person
- Tool

Activity, phase

- Activity
  - Time passed by resource to perform defined, coherent task
- Phase
  - Set of activities

Milestone

- Key event/condition in the project
- with effects on subsequent activities
- ex. requirement document accepted by the customer
  - if yes then ..
  - if no then ..

Milestones example

- M0: contract signed
- M1:Requirements review passed
- M2: Design review passed
- …

Milestones example

- M1:Requirements document + UI prototype delivery
- M2: Design document delivery
- …

Deliverable

- Product (final or intermediate) in the process
  - Cfr requirements document, prototype
- internal (for producer) or external (for customer)
- contractual value or not

Deliverables

- Ex in course project
  - Requirements document
  - Estimation document
  - Design document
  - Code
  - Test cases (unit, integration, acceptance)
  - Time sheet

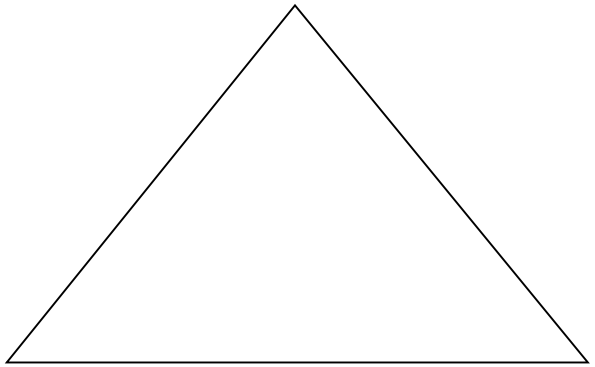
Measures

Relevant measures

- Process measures
  - time, effort, cost
  - productivity
  - earned value
  - fault, failure, change
- Product measures
  - Functionality (FP)
  - Size
  - Price
  - Modularity
  - Other .. ilities

Project Management

Software System (functions and quality)



Calendar time Cost

No notion of unpredictable events here

Calendar time, or duration

- Days, weeks, months, on calendar
- Relative, from project start
  - Month1, month2, etc
  - Typically used in planning
- Absolute
  - September 12
  - Typically used in controlling
  - Remark, transition relative -> actual is not 1 to 1 (vacations, etc)

Relative to absolute

- Ex: project requires one calendar month (relative, from project start)
  - Absolute:
    - If start is August 1, and everybody is on vacation until August 31, end is September 30
    - If start is August 1, and everybody is available, end is August 31

Effort

- time taken by a resource to complete a task
- Depends on duration and on #resources employed

  duration * resource
- Measured in person hours (ieee 1045)
  - person day, person month, person year depend on national and corporation parameters

Effort

- 1 person works 6 hours → 6 ph
- 2 persons work 3 hour → 6 ph
- 6 persons work 1 hour → 6ph
- 2 persons work ½ hour → 1ph

Typical conversions

- 1 person day = 7 ph
- 1 person week = 35 ph
- 1 person month = 140 ph
- 1 person year = 1680 ph

WARNING : these conversions are NOT valid everywhere (depend on companies and nations)

Examples

- Company A, Italy, person week = 38 ph
- Company B, Italy, person week = 40 ph
- Company C, USA, person week = 42 ph
- Company D, France, person week = 35 ph

- etc

Calendar time vs. effort

- Always linked
- Mathematical link. 6 ph can last
  - 6 calendar hours if 1 person works
  - 3 calendar hours if 2 persons work in parallel
  - 1 calendar hour if 6 persons work in parallel
- Practical constraint
  - Is it feasible?
    - One woman makes a baby in 9 months
    - 9 women make a baby in one month?

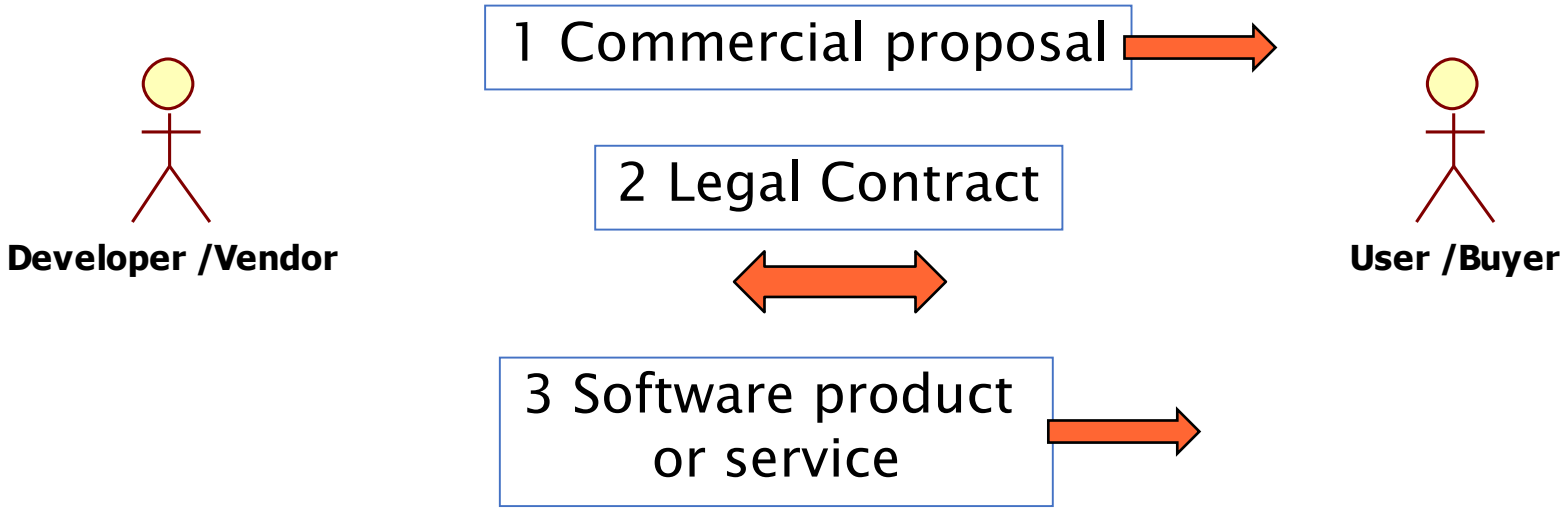
Effort and cost

- Effort can be converted in cost, knowing cost per ph
  - Staff cost = person hours * cost per hour

Variants of measure

- Estimated value: what is forecasted
  - Ex estimated duration for project X: 30 days
- Actual value: what is real
  - Ex actual duration for project X: 35 days
- Target value: what is desirable
  - Ex target duration for project X : 28 days
- Benchmark: what the others do
  - Ex benchmark duration project X: 29 days

Costs and roles



Cost - vendor

- Personnel
  - Staff
    - Person hours, salary
    - Overhead costs (office space, heating/cooling, telephone, electricity, cleaning, ..)
  - Hardware
    - Development platform, (target platform)
  - Software
    - Licenses (OS, DB, tools ..)

Cost - user

- Total Cost of Ownership (TCO)
  - Considers the complete time window involving the product
  - At least three phases
    - Before acquisition
    - Usage
    - Dismissal

Cost – user (2)

- Before acquisition
  - Costs to define requirements and select the product
    - Market analysis, feasibility studies, requirement definition, vendor / product evaluation, contract negotiation
- Acquisition
  - Acquisition cost
    - one time fee, yearly fee, usage fee
  - Acquisition cost (= price) ⇔ vendor cost + profit

Cost – user (3)

- After acquisition
  - Deployment costs
    - Install in all users machines
    - Training for users
    - Learning curve
  - Operation costs
    - Servers, network
  - Maintenance costs
    - Collection of anomalies, effect of anomalies
    - Corrective, evolutive, enhancement maintenance

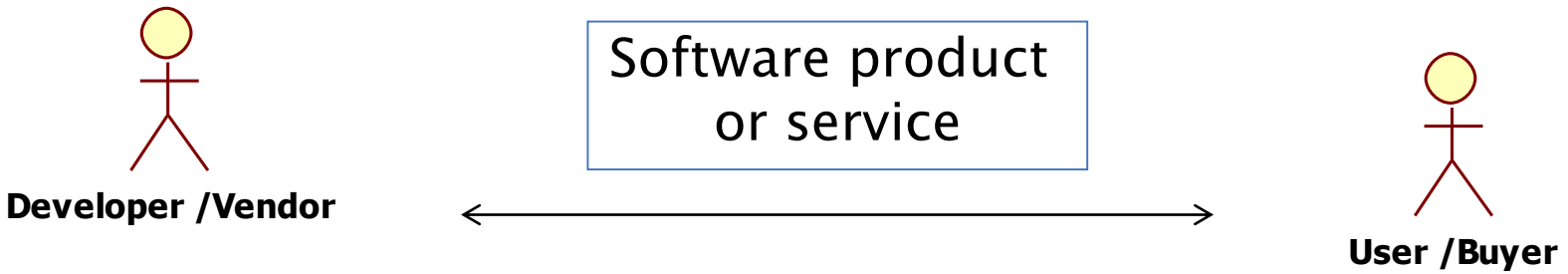
Cost – user (4)

- Dismissal
  - Uninstall product
  - Back up data, data conversion ..

TCO

- The longer the time frame, the less important the acquisition cost
  - Ex, commercial airplane
  - Time frame: 20 years (50.000 hours)
  - Acquisition cost of airplane = 1/6 of TCO
    - Key TCO cost factors are fuel, crew, maintenance

Cost and price



- Cost = cost for vendor
- Price = acquisition cost for user
  - In the simplest model price = cost + margin
  - However, in the general case any relation is possible between cost and price

Size

- Of source code
  - LOC (Lines of Code)
- Of documents
  - Number of pages
  - Number of words, characters, figures, tables
- Of test
  - N test cases

Size

- Of entire project
  - Function points (see later)
  - LOC
    - In this case LOCs virtually include all documents (non code) produced in the application
    - Ex. project produces 10 documents (1000 pages) and 1000 LOCs. By convention project size is 1000 LOCs

LOC

- What to count
  - w/wout comments
  - w/wout declarations
  - w/wout blank lines
- What to include or exclude
  - Libraries, calls to services etc
  - Reused components
- Comparison for different languages not meaningful
  - C vs Java? Java vs C++? C vs ASM?

LOC – what to include exclude

- Depends on the goal of counting
  - For cost estimation: count only what should be developed
  - For memory occupation: count everything
  - For price: consider whole functionality offered

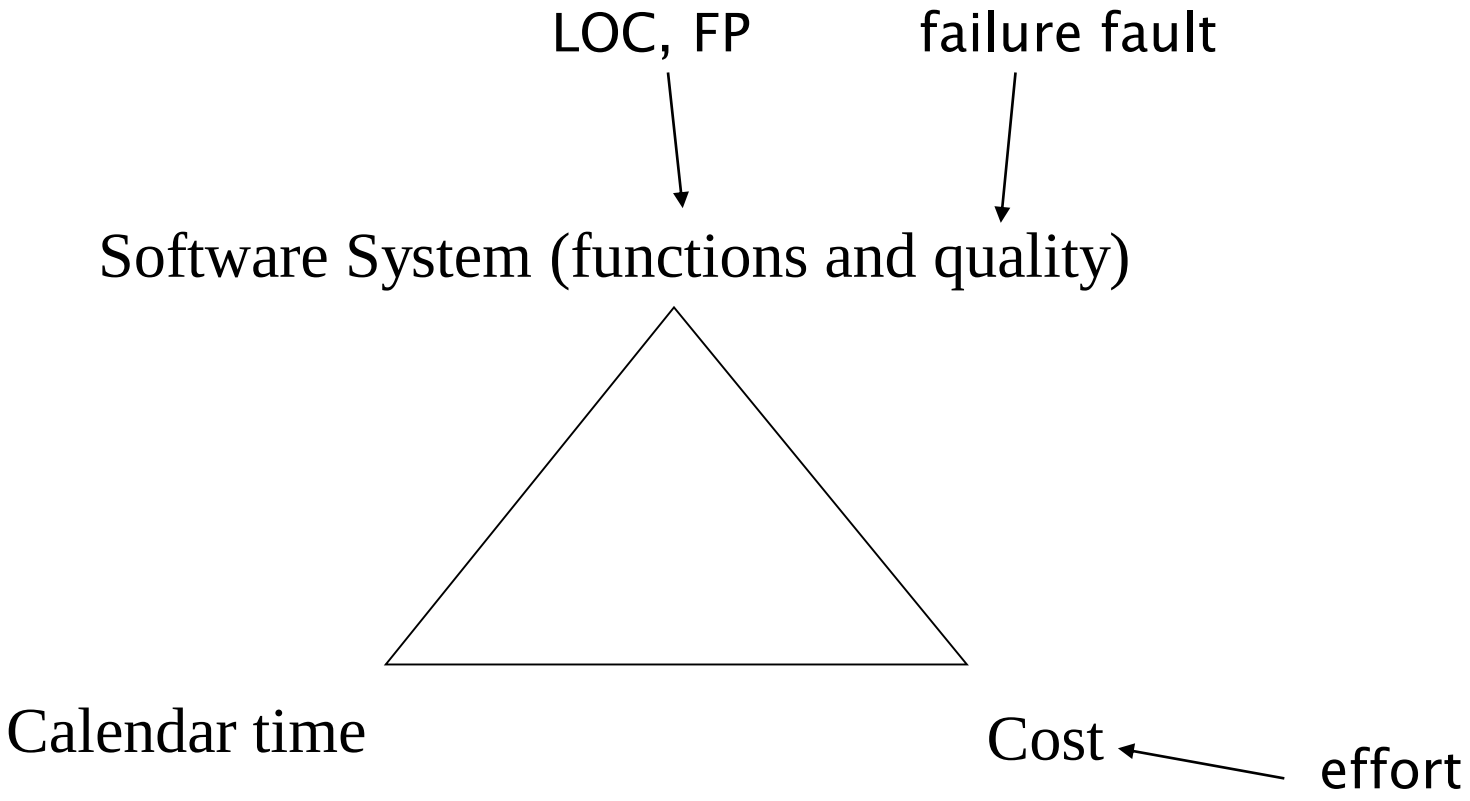
Productivity

- Output/effort
- What is output in software?
  - Size/effort = LOC / effort
    - Inherits problems of LOC
  - Functionality/effort = FP/effort
  - Object Points / effort

LOC/effort

- The lower level the language, the more productive the programmer
  - The same functionality takes more code to implement in a lower-level language than in a high-level language
- The more verbose the programmer, the higher the productivity
  - Measures of productivity based on lines of code suggest that programmers who write verbose code are more productive than programmers who write compact code

PM and Measures

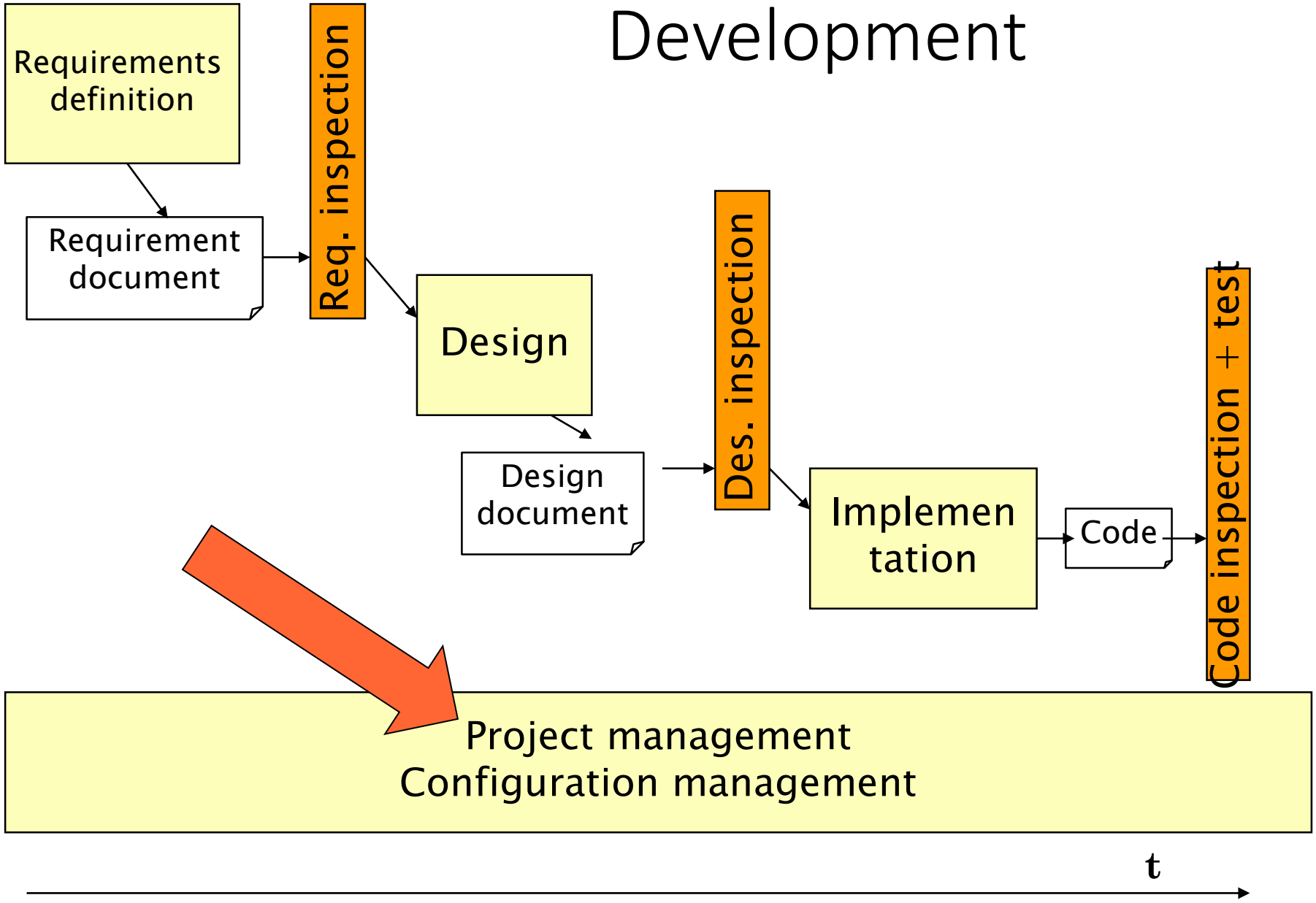


Productivity figures

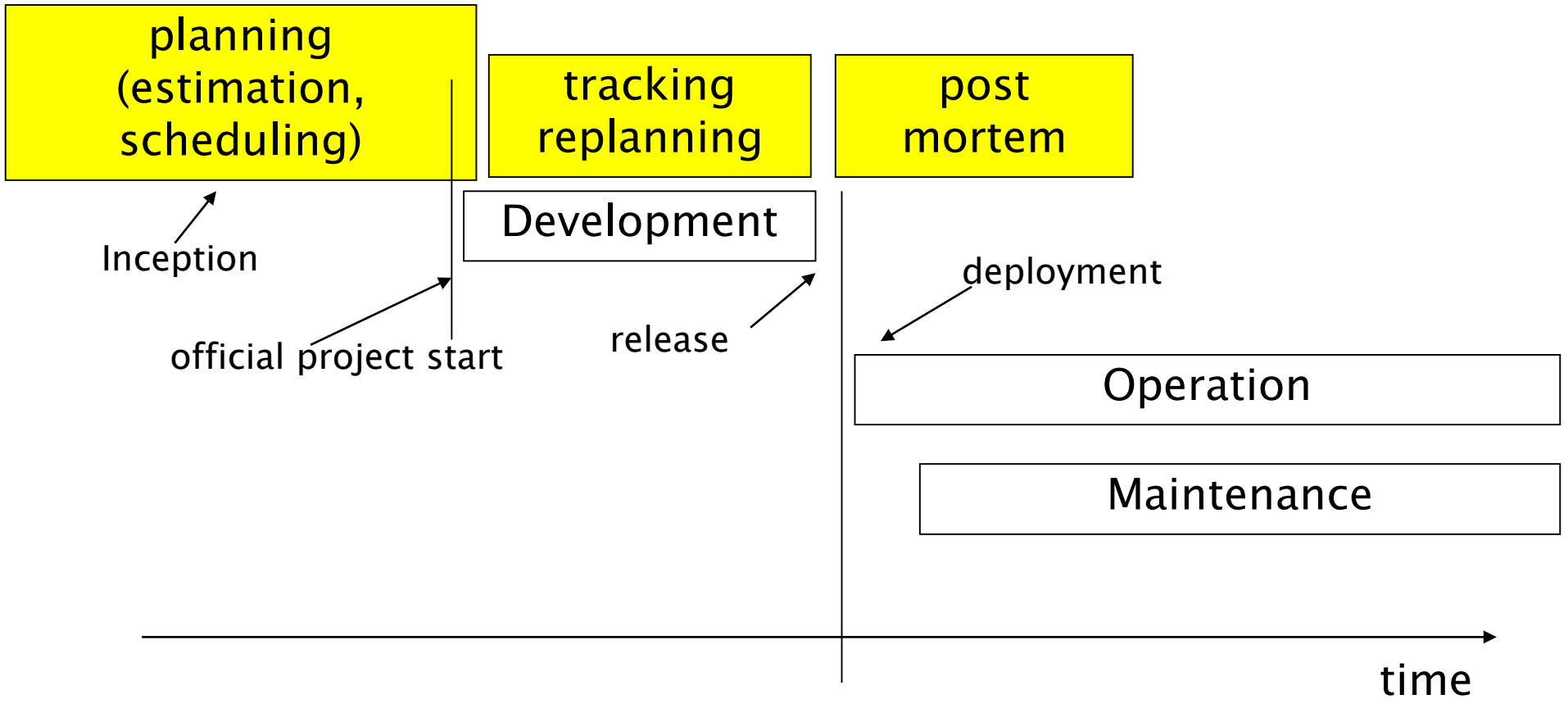
- Real-time embedded systems, 40-160 LOC/P-month
- Systems programs, 150-400 LOC/P-month
- Commercial applications, 200-800 LOC/P-month

- Source: Sommerville

Development



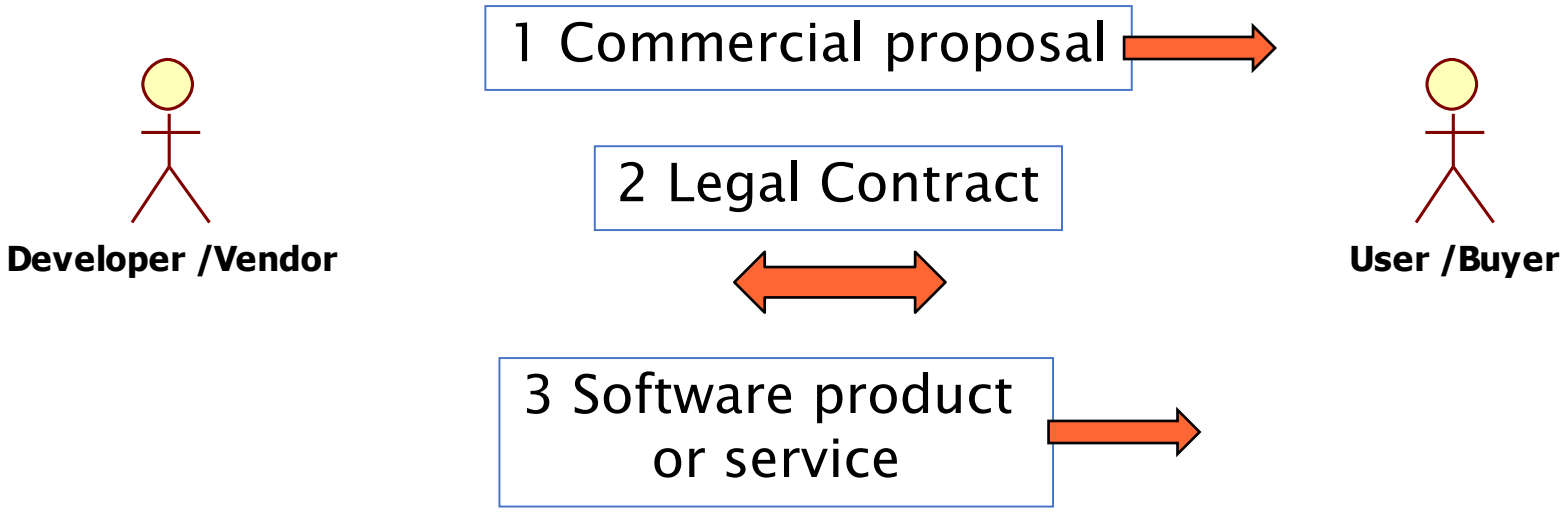
PM activities



PM - inception

- Projects do not start ‘in zero time’
- **Inception** phase
  - Initial analysis of requirements
  - Initial general architecture
  - Initial estimate of duration and cost
  - Commercial proposal

The vendor / buyer relationship



PM - inception

- Not all projects pass the inception phase
- Inception phase has a cost
  - Normally not paid to vendor
  - In very large projects may be paid, as ‘concept development’ or ‘feasibility study’

Common issues

- Vendor tends to overpromise (or underestimate cost and time) to gain the contract
  - Counting on variations (and related costs / gains) to be made later
  - Counting on gains in maintenance / operation phase (buyer is typically locked in to vendor for maintenance)

Estimation

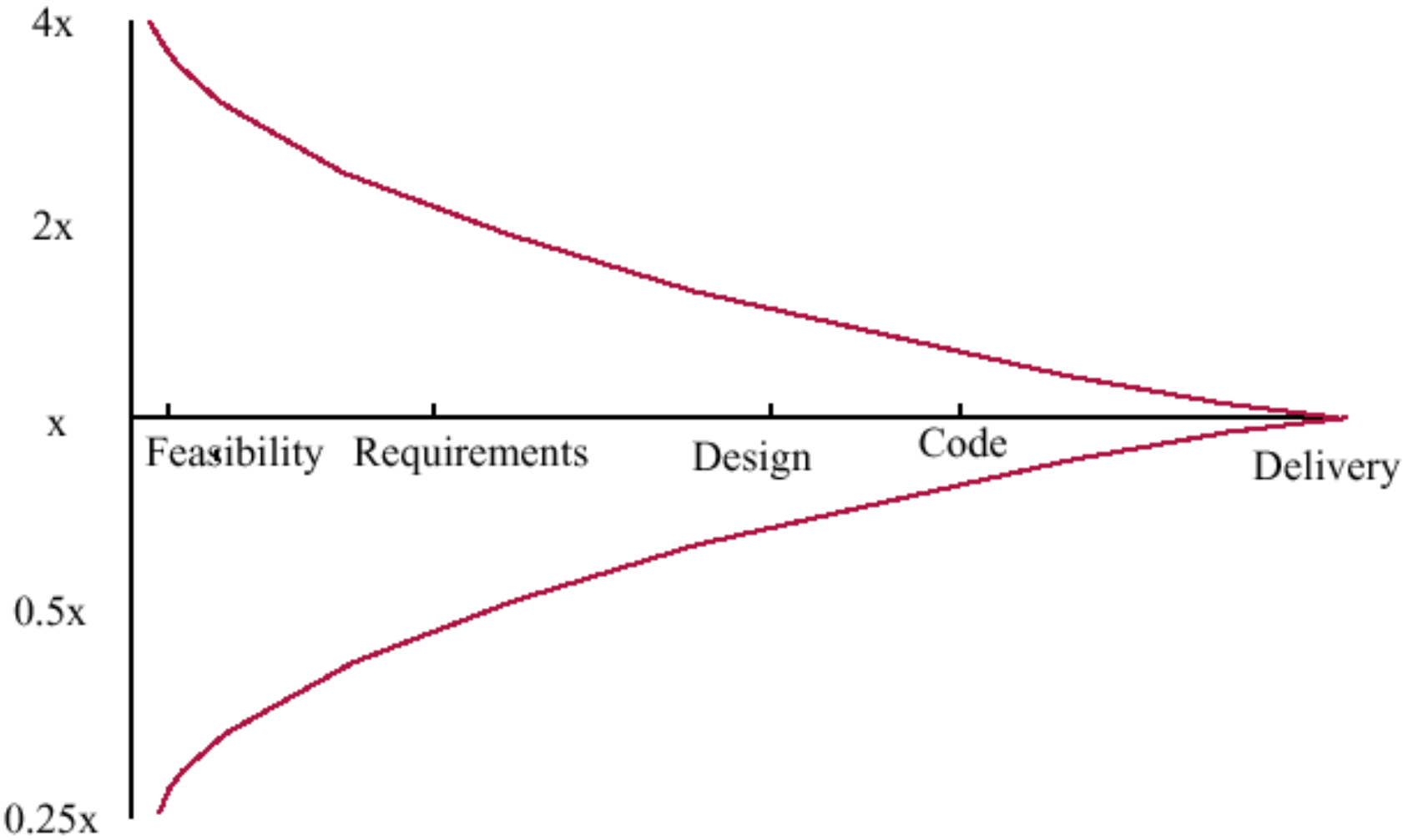
Estimation of cost and effort

- Based on analogy
  - requires experience from the past to ‘foresee’ the future
    - Experience can be qualitative (in mind of people) or quantitative (data collected from past projects)
  - the closer a project to past projects, the better the estimation

Estimation accuracy

- The cost/effort/size of a software system can only be known accurately when it is finished
- Several factors influence the final size
  - Use of COTS and components
  - Programming language
  - Distribution of system
- As the development process progresses, then the estimate becomes more accurate

Estimate uncertainty



Techniques

- (Parkinson, pricing to win)
- Based on judgment
  - Decomposition
    - By activity (WBS)
    - By products (PBS)
    - Gantt chart
  - Expert judgement, Delphi
- Based on data from the company
  - Size and productivity (Regression models)
  - Analogy, case based
- Based on models
  - Function points
  - Cocomo, Cocomo2
  - Object points

Parkinson's Law

- The project costs whatever resources are available
- Advantages: No overspend
- Disadvantages: System is usually unfinished

Pricing to win

- The project costs whatever the customer has to spend on it
- Advantages: You get the contract
- Disadvantages: The probability that the customer gets the system he or she wants is small. Costs do not accurately reflect the work required

By decomposition

- By activity
  - Identify activities (WBS)
  - Estimate effort per activity
  - Aggregate (linear)

- By product
  - Identify products (PBS)
  - Estimate effort per product
  - Aggregate (linear)

- Rationale: easier to estimate smaller parts

WBS

- Work Breakdown Structure
- Hierarchical decomposition of activities in subactivities
- no temporal relationships

WBS example

- Requirements planning
  - Review existing systems
  - Perform work analysis
  - Model process
  - Identify user requirements
  - Identify performance requirements
  - …
- Design
  - …
- Implementation
  - …

PBS

- Product Breakdown Structure
- hierarchical decomposition of product
  - Product
    - Requirement document
    - Design document
    - Module 1
      - Low level design
      - Source code
    - Module 2
      - Low level design
      - Source code
    - Testdocument

WBS vs. PBS

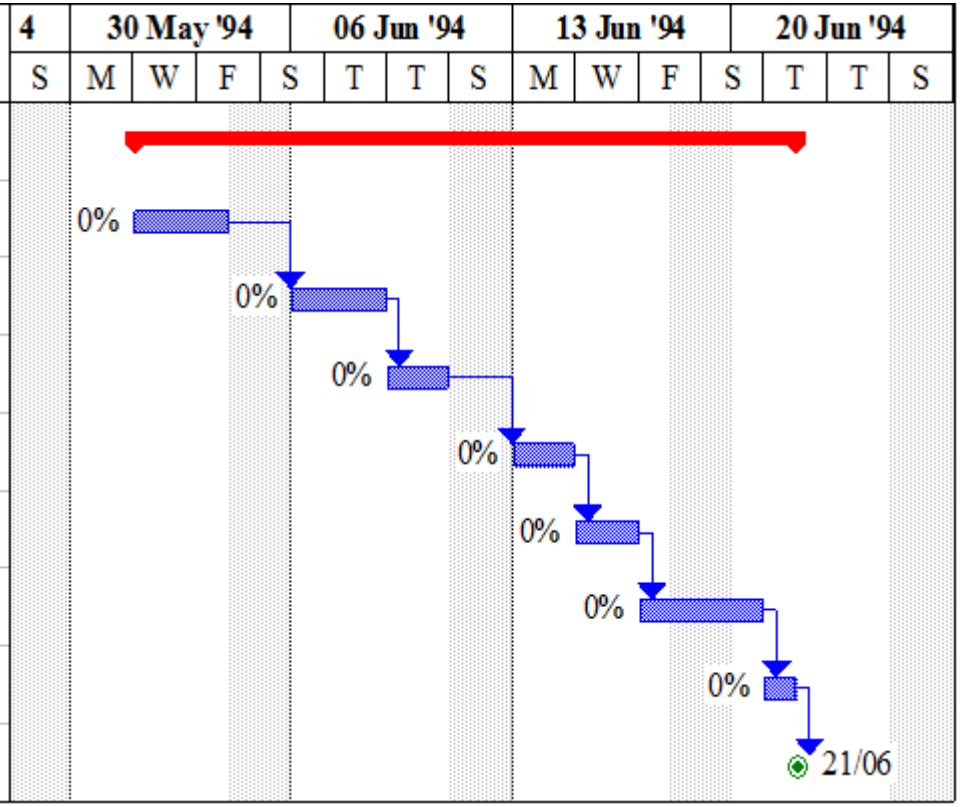
- WBS (Work Breakdown Structure)
  - Answers: “What needs to be done?”
  - Breaks down the necessary activities to complete the project.
- PBS (Product Breakdown Structure)
  - Answers: “What do we need to deliver?”
  - Breaks down the final product into parts/components.
- Using Both Together
  - Define the PBS first for a clear overview of all deliverables.
  - Create the WBS next, linking each product component to the specific tasks required, assigning time, cost, and responsibility.

Gantt chart

- Improvement on estimate by WBS
  - Better duration estimate
- Activities from WBS are placed on a calendar
  - Parallel vs serial activities are spotted

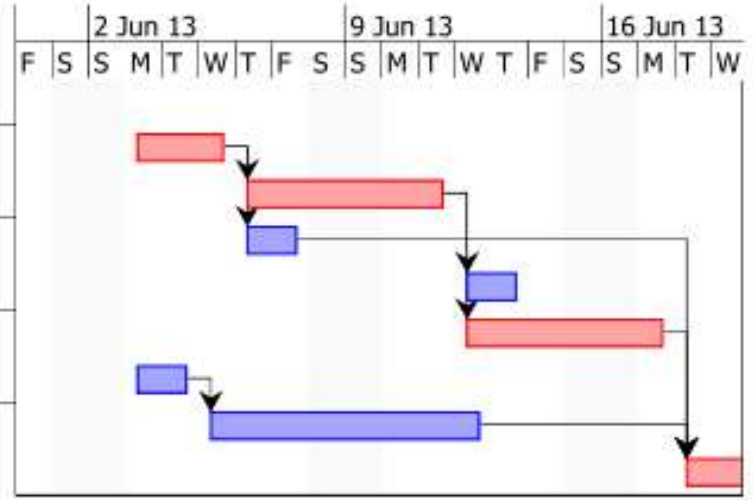
Gantt chart

ID	Task Name
1	**Requirements Planning**
2	Review existing systems
3	Perform work flow analysis
4	Model process
5	Identify user requirements
6	Identify performance requirements
7	Identify interface requirements
8	Prepare Software Requirements Specification
9	**Software Requirements Review**



Gantt

Name	Duration	Start	Finish	Predecessors	Resource Names
Requirements	3 days	6/3...	6/5/1...		
Design	4 days	6/6...	6/11/...	1	
Test data	2 days	6/6...	6/7/1...	1	
Documentation	2 days	6/1...	6/13/...	2	
Coding	4 days	6/1...	6/17/...	2	
Test plan	2 days	6/3...	6/4/1...		
Unit test	6 days	6/5...	6/12/...	6	
System test	4 days	6/1...	6/21/...	5;3;7	



Expert judgement

- one (or more) experts (chosen in function of experience) propose an estimate
- Similar to estimate by decomposition, but estimates are made by expert(s) in the domain not involved later in development
  - instead of being made by developer(s)

  - Assumption: experts have better knowledge, due to experience from participation to several projects in similar domain

Ex: requirements

Activity	Estimated effort (person days)
Review existing systems	5
Perform work analysis	5
Model process	1
Identify user requirements	10
Identify performance requirements	4
TOTAL	25

Estimation by size and productivity

- Step 1: estimate total size of project (Loc) → S
- Step 2: apply productivity rate P to obtain estimated effort
  - P = S / effort
  - Effort = S / P
- Step 3: apply cost rate to obtain estimated cost
  - Cost = effort * cost rate

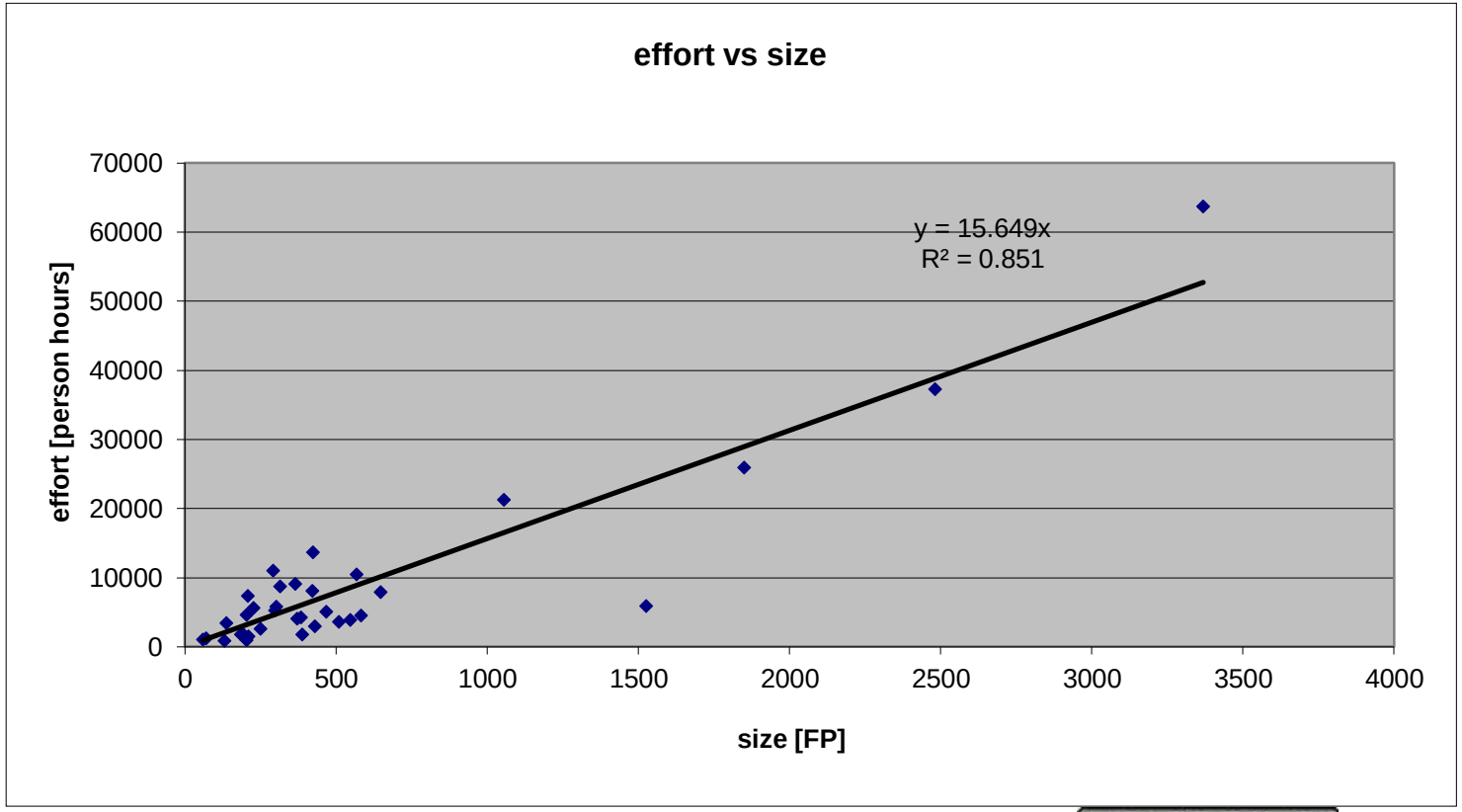
Example: estimated size = 50.000 Loc
productivity rate = 50 Loc /person hour
estimated effort = 50.000 / 50 = 1000 person hours
cost rate = 35 euro / person hour
estimated cost = 1000 * 35 = 35.000 euros

Estimation by size and productivity

- Productivity rate:
  - Not one value, it depends on many factors
    - Personnel involved, technologies used, software type and criticality, domain
    - It should come from company data
  - It includes effort for **all** activities in the software process (not only coding)
- Cost rate:
  - Depends on personnel involved and company
- Size:
  - better to estimate it considering smaller components (ex classes, packages)
    - Ex 50.000 Locs could be the result of estimating a size of 500 classes, 100 Locs each

Estimation by size and productivity

- Example of how to estimate productivity
  - Each data point is a project completed in a company
  - For each project size and effort are recorded
  - The regression line provides the productivity estimate
  - For each project effort includes effort for ALL activities (requirement, design coding etc) as collected via time sheets



Estimation by size and productivity

- Step 4: estimate calendar duration
  - A very rough estimate can be obtained by dividing estimated effort by the number of staff working
    - Ex: estimated effort = 1000 person hours
      number of staff working full time on project: 4
      estimated calendar days = 1000 / 4 = 250
  - The estimate is unprecise (optimistic) because not all activities in the project will be feasible by all staff members (250 is a lower bound)
  - A gannt chart would provide a more precise estimate

Function Points

- A metric for software project estimation.
- To measure the size of software based on its functionality.
- Developed by Allan Albrecht at IBM in 1979.
- to estimate time, cost, and effort in software development.

Function Points

- Independent of programming language.
- Based on "user view" - what the user requests and receives.
- Helps compare and track productivity across different teams/languages.
- Useful for managing both development and maintenance projects.

Function Points

$$fp = A*EI + B*EO + C*EQ + D*EIF + E*ILF$$

- **External Inputs (EIs)** - Processes that data comes into the system.
- **External Outputs (EOs)** - Data outputs from the system.
- **External Inquiries (EQs)** - Interactive transactions made by users.
- **External Interface Files (EIFs)** - External systems data used by the application.
- **Internal Logical Files (ILFs)** - User identifiable groups of logically related data.

Function Points

- Procedure:
  - Identify and classify components as ILFs, EIFs, EIs, EOs, or EQs.
  - Assign each component a complexity weight (Low, Average, High).
  - Use weighting tables to allocate points based on complexity.
  - Sum all the points to get the total Function Points.

- Coefficients A,B,C,D,E

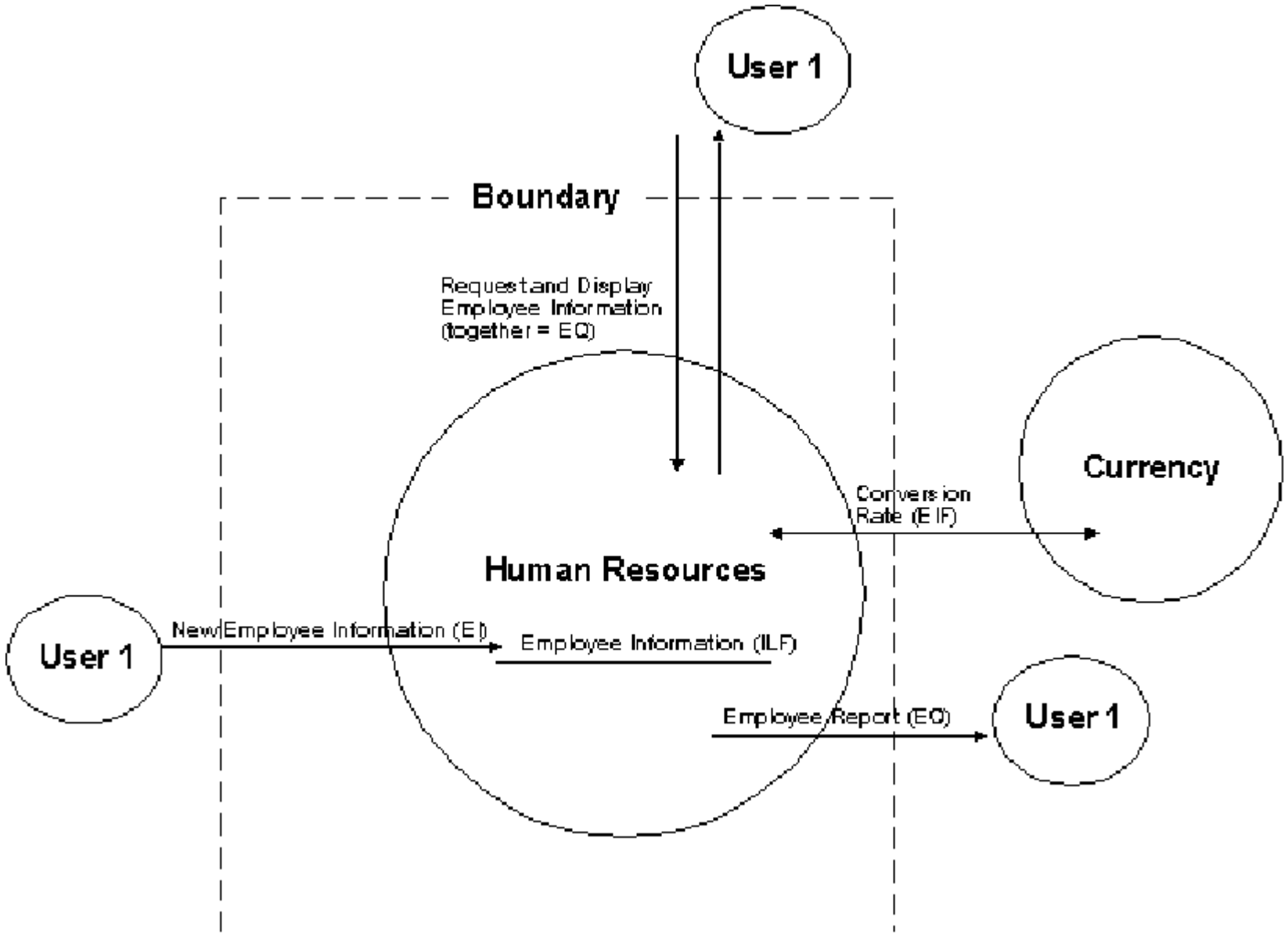
	Level of Complexity		
Component	**Simple**	**Average**	**Complex**
Input item	3	4	6
Output item	4	5	7
Inquiry	3	4	6
Master file	7	10	15
Interface	5	7	10

Function Points

- For any product, size in “function points” is given by

$$FP = 4 \times EI + 5 \times EO + 4 \times EQ + 10 \times ILF + 7 \times EIF$$

- A 3-step process.


User 1
Boundary
Request and Display
Employee Information
(together = EQ)
Currency
Conversion
Rate (EIF)
Human Resources
User 1
New Employee Information (EI)
Employee Information (ILF)
Employee Report (EO)
User 1

Technical Complexity Factor (TCF)

- TCF adjusts the Function Points to reflect the technical complexity and the technological challenges encountered in the project.
- It accounts for computational performance, reusability, and data processing needs.

TCF

- **Data Processing Complexity**: Impacts of extensive data management and manipulation.
- **Performance Requirements**: The need for high-performance metrics under specific constraints.
- **End-User Efficiency**: Focus on user interface and experience optimizations.
- **Complex Processing**: The requirement for complex algorithms or computing processes.
- **Reusability**: Designing software components that can be reused in other applications.
- **Installation Ease**: Complexity involved in software deployment and installation.

Environmental Complexity Factor

- ECF adjusts the Function Points to consider the project's environmental variables like team experience, working conditions, and project continuity.
- It reflects the non-technical influences that can affect the project timelines and efficiency.

ECF

- **Team Experience**: Level of experience the development team has with similar projects.
- **Application Experience**: Familiarity with the technology and the application domain.
- **Team Motivation**: Impact of team morale and motivation on the project's progress.
- **Working Conditions**: The work environment's adequacy for efficient project execution.
- **Required Software Tools**: Availability and adequacy of tools to support the project development.

- TCF=0.6+(0.01×$\sum$ Technical Factor)
- ECF =1.4+(−0.03 × $\sum$ Environmental Factor)

Calculating Adjusted Function Points

$$AFP = UFP \times TCF \times ECF$$

- Adjusted Function Points provide a more accurate measure of the project's scope by incorporating adjustments for technical and environmental complexities.

FP

- Advantage
  - Independent of technology
  - Independent of programmer
  - Well established and standardized
- Downside
  - Counting long and expensive
  - Transaction system oriented (no real time, no embedded systems)

FP vs. LOCS

	FP	LOCs
Depend on prog language	N	Y
Depend on programmer	N	Y
easy to compute	N, must be done by trained person	Y, tool based (after end of project)
Applicable to all systems	N, transaction oriented	Y

FP as unit of exchange

- Company A bids for FP
  - Buy 10000 FP, how much? (bid)
  - providers answer, x Euro per FP
- A selects provider
  - lowest cost and other factors
- End of year, redo counting
  - 10123 FP actually delivered
  - A pays

Reminder

- Measures of size
  - FP, LOC
- Both can be computed
  - Before a project start (estimated size)
  - After a project ends (actual size)
- Both can be used to
  - Characterize productivity
    - FP/effort, LOC/effort
  - Characterize application portfolio
    - FP or LOC owned and operated by a company

PM approaches

- Two radically different approaches
  - Depend on process model chosen: ‘waterfall’ vs ‘iterations’
- Traditional, waterfall
  - Requirements → project plan
- Time framed, agile
  - ‘Dynamic’ requirements, iterations

Waterfall	Iterations
Define requirements	1 Define requirements (less deeply)
Estimate effort for all requirements	2 Rank requirements
Define Gantt (one iteration)	3 Pick top requirements feasible in one iteration (4 weeks)
Develop	Repeat 2,3 until finished

Ex waterfall

- 1 Define requirements
  - F1, F2, F3, F4, F5, F6, F7
- 2 Estimate
  - 120 person days
- Define Gantt
  - 3 people available
  - Roughly 40 calendar days needed
- Develop

Ex iterations

- 1 Define requirements
  - F1, F2, F3, F4, F5, F6, F7
- 2 rank
  - F7 (20person/day) , F4 (25person/day) , F2 (15), F5 (10), F1 (20), F3 (10), F6 (20)
- 3 Pick requirements for one iteration

  ex 3 people available x 4 weeks

  = 3x4x5 = 60 person days available

- 4 do iteration 1
  - Deliver F7, F4, F2

- Repeat ranking – requirements can change

Comparison

- In theory in both cases F1 to F7 in 40 calendar days
- In practice Iterations produce a different result

Comparison

- Waterfall: suitable for stable environments, larger projects, dependencies on hardware, safety critical
  - Ex automotive, aerospace
- Iterative: suitable for dynamic environments, smaller projects

Planning

Planning Process

- Identify activities and/or deliverables
  - PBS, WBS
  - reference models (CMM, ISO12207)
- estimate effort and cost
- define schedule (Gantt)
- analyze schedule (Pert)

Project plan

- living document
  - will be updated during tracking
- outline
  - list of deliverables, activities
  - milestones
  - Gantt
  - Pert
  - personnel organization
  - roles and responsibilities

Scheduling

Project duration

- As well as effort estimation, calendar time must be estimated, and staff allocated
- Scheduling can be done on Gantt/Pert
- COCOMO2 gives also an estimate of calendar time
  - Independent of staffing

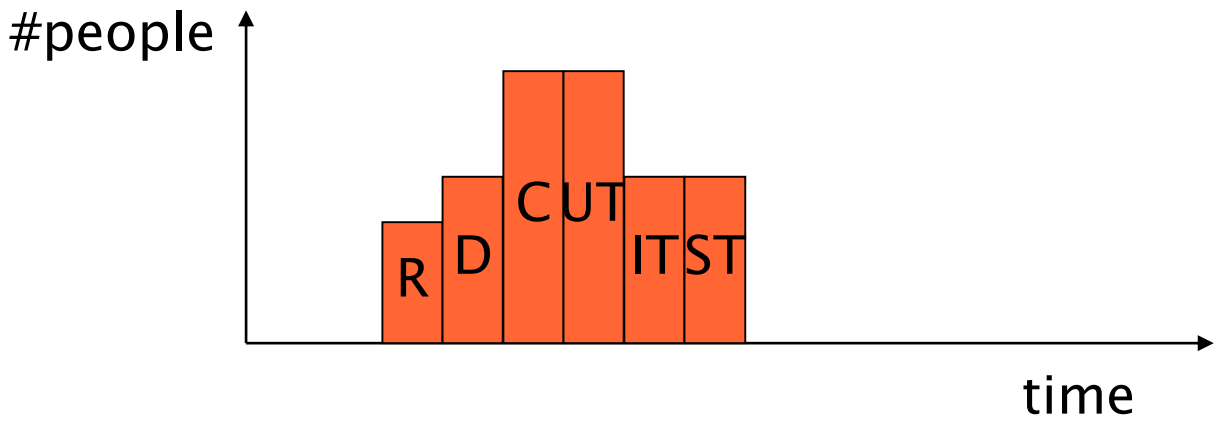
- Calendar time can be estimated using a COCOMO 2 formula
  - $TDEV = 3 \times (PM)^{(0.33+0.2*(B-1.01))}$
  - PM is the effort computation and B is the exponent computed as discussed above (B is 1 for the early prototyping model). This computation predicts the nominal schedule for the project

Staffing requirements

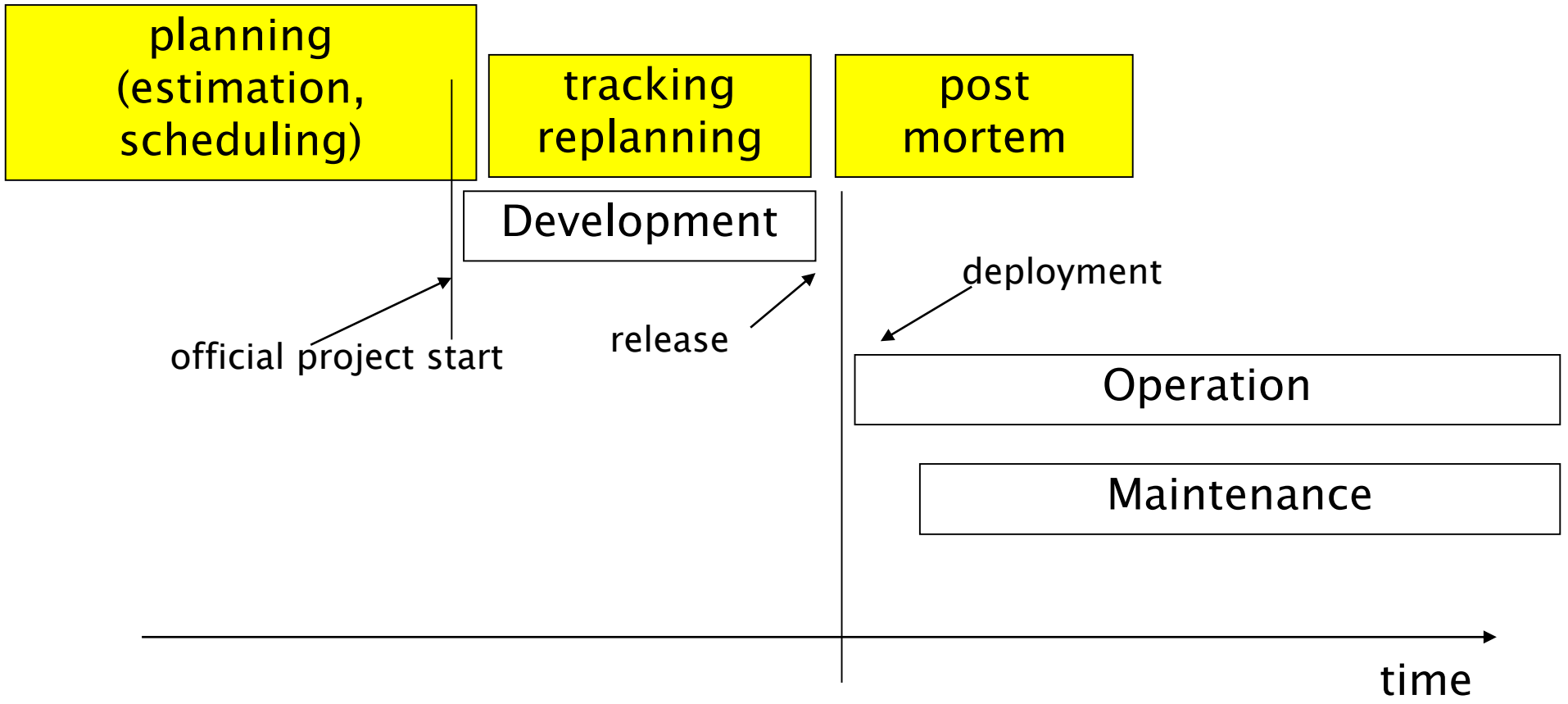
- Staff required can’t be computed by dividing the development time by the required schedule.
- The number of people working on a project varies depending on the phase of the project.
- The more people who work on the project, the more total effort is usually required.
- A very rapid build-up of people often correlates with schedule slippage.

Staffing profile

- Number of people working on the project vs. time
- Typically has a bell shape
  - duration of project is constrained by staffing profile + total effort estimated



The PM activities



Tracking

Tracking process

- The project has started - how to know the status of the project?
- collect project data, define the actual status
- compare estimated – actual
  - Estimated Gantt is the roadmap for the project
- if deviations, do corrective actions
  - change personnel, change activities, change deliverables, ...
  - re-plan, update Gantt and PERT

Time sheet

- Basic tool to collect actual effort spent by employees
  - Per day, Per project, per activity in project
- Allows to collect
  - Actual effort spent per project
- To track project status
- To compute cost of project

Project status

- Option1
  - Effort spent
- Option2
  - Effort spent + activities closed
- Option3
  - Earned value

Effort spent

- Collect effort spent, compare with estimated
  - Ex, spent 10, estimated 100, we are done 10%
- Big flaw, confounds input measure (effort spent) with output measure (completion)

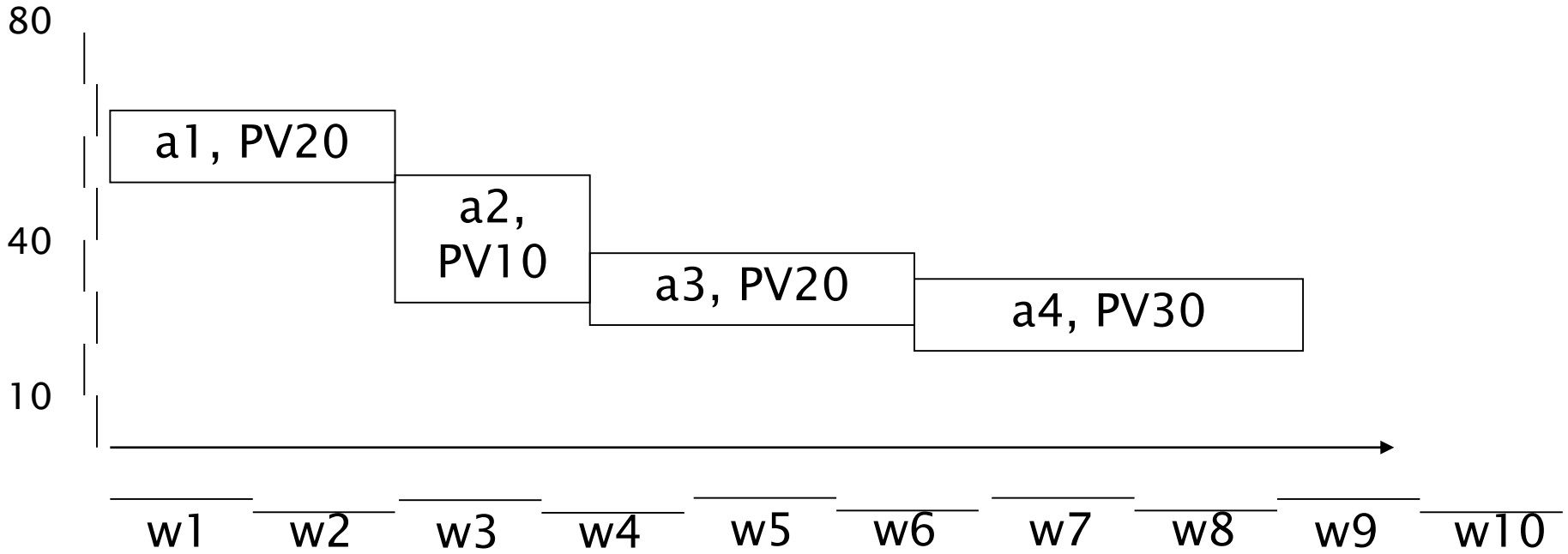
The first **90**% of a project takes **90**% of the time, the last 10% takes the other **90**% of the time
[Murphy's law]

Activities closed

- How to define when activity is closed?
  - All effort planned for activity is spent
    - Same problem, confounds input with output
  - Define quality gate, level to achieve
    - Ex, requirements: inspection meeting, majority of participants judges document is goodenough
    - Ex, unit testing: coverage 95% of nodes, and all tests pass

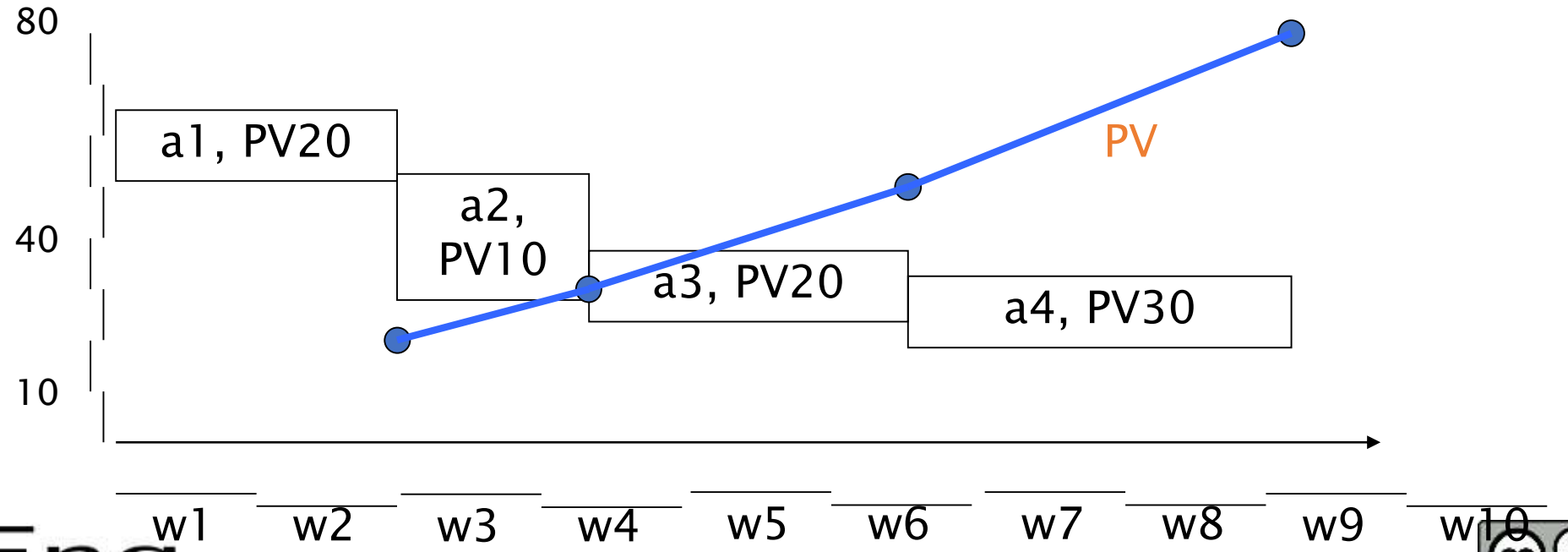
Earned value

- A technique to measure the progress of a project
- Step 1: identify activities, assign a value to them (Planned Value, PV), schedule them



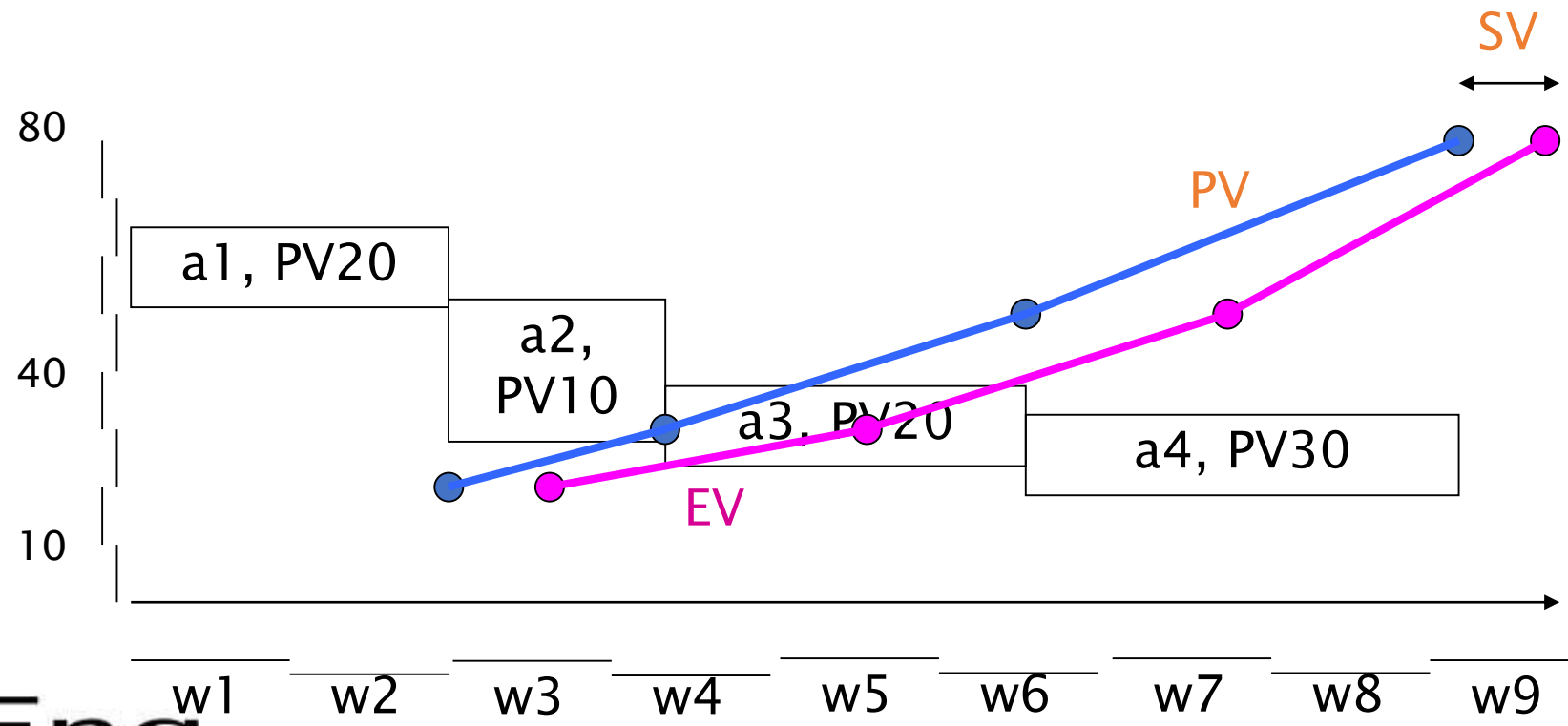
Earned value

- Step 2: define a rule to pass from PV to EV (rule1 0/100 or rule2 0/50/100)
  - With the rule1, the project earns the PV of an activity when the activity is 100% finished.



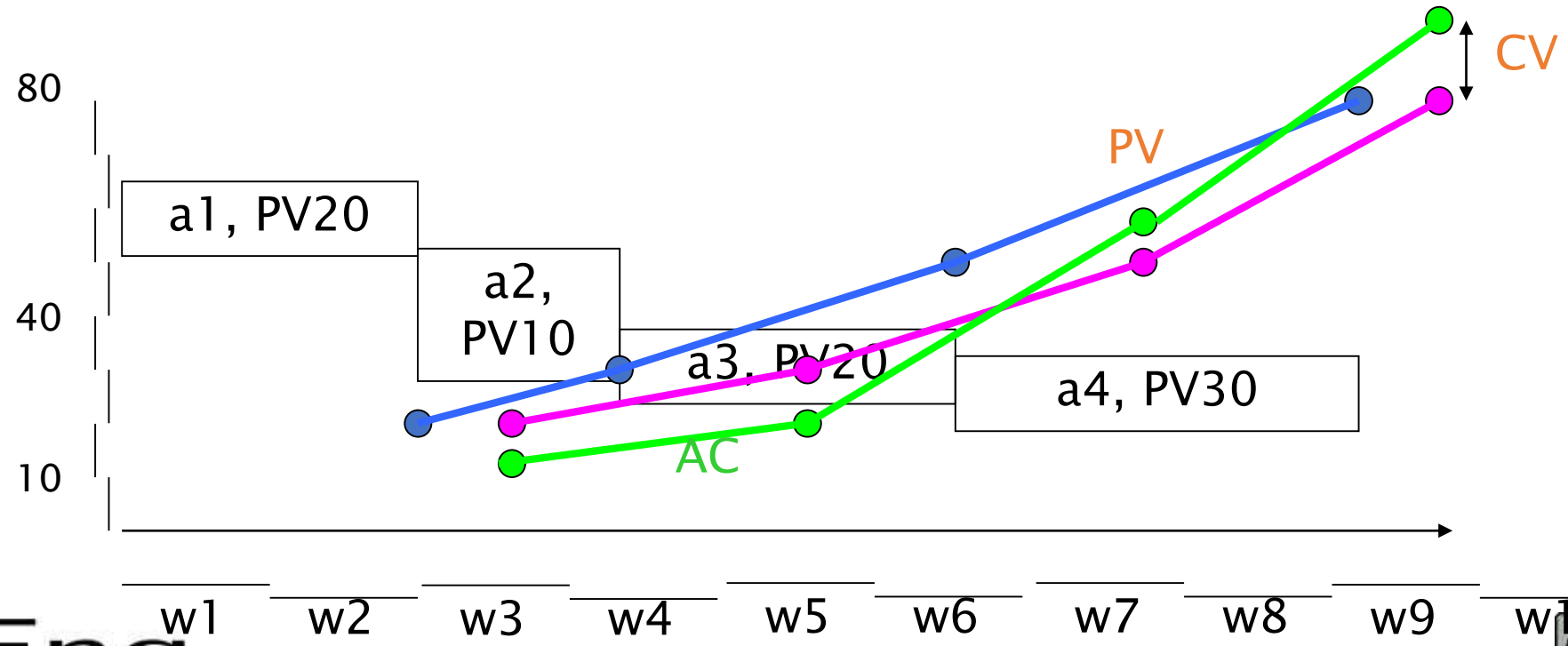
Earned value

- Step 3: start the project, measure EV and compare with PV

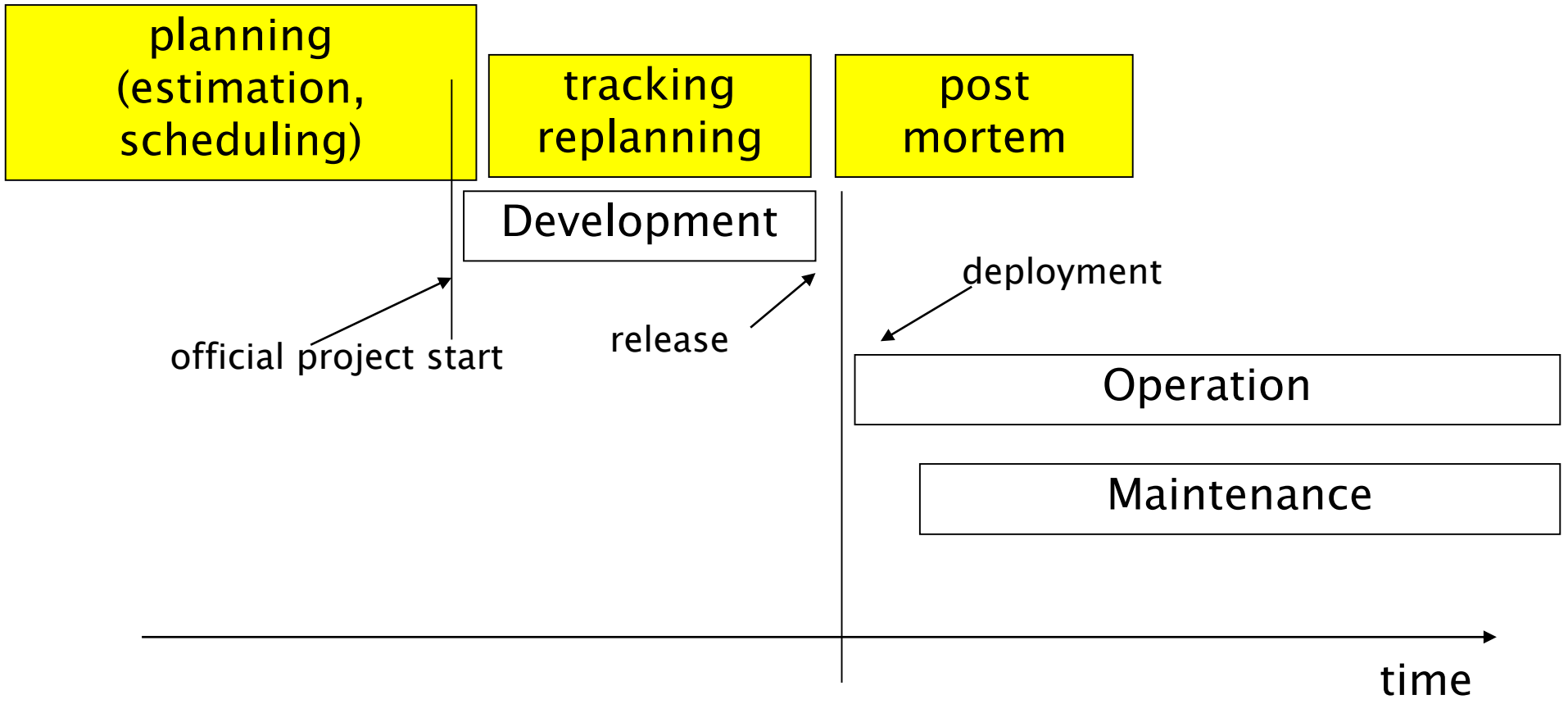


Earned value

- Step 4: compute also AC, actual cost



The PM activities



Post Mortem

Post mortem

- A form of organizational learning
- Collect key information from the project
  - Effort, faults – estimated and actual
  - Achievements
  - Problems and causes
- To make it available to other projects

PMA – learn from experience

- PMA (when used appropriately) PMA ensures that team members recognise and remember what they learned during a project.
- PMA identifies improvement opportunities and provides means to initiate sustained change.
- PMA provides qualitative feedback
- Two types
  - General PMA
  - Focused PMA – understanding and improving a project`s specific activity

PMA process

- Preparation
  - Study the project history to understand what has happened
  - Review all available documents
  - Determine goal for PMA
  - Example of goal: Identify major project achievements and further improvement opportunities.

PMA process cont.

- Data collection
  - Gather relevant project experience
  - Focus on positive and negative aspects
  - Semistructured interviews – pre-prepared list of questions
  - Facilitated group discussion
  - KJ sessions
    - Write down up to four positive and negative project experience on post-it notes.
    - Put the notes on a whiteboard
    - Re-arrange notes into groups and discuss them

PMA process cont.

- Analysis
  - Feedback session
    - Have we (analyser) understood what you (project member) told us, and do we have all the relevant facts?
  - Ishikawa diagram in a collaborative process to find the causes for positive and negative experiences
    - Draw an arrow on a whiteboard – which is label with experience
    - Add arrows with causes (the diagram will look like a fishbone)

PMA – results and experience

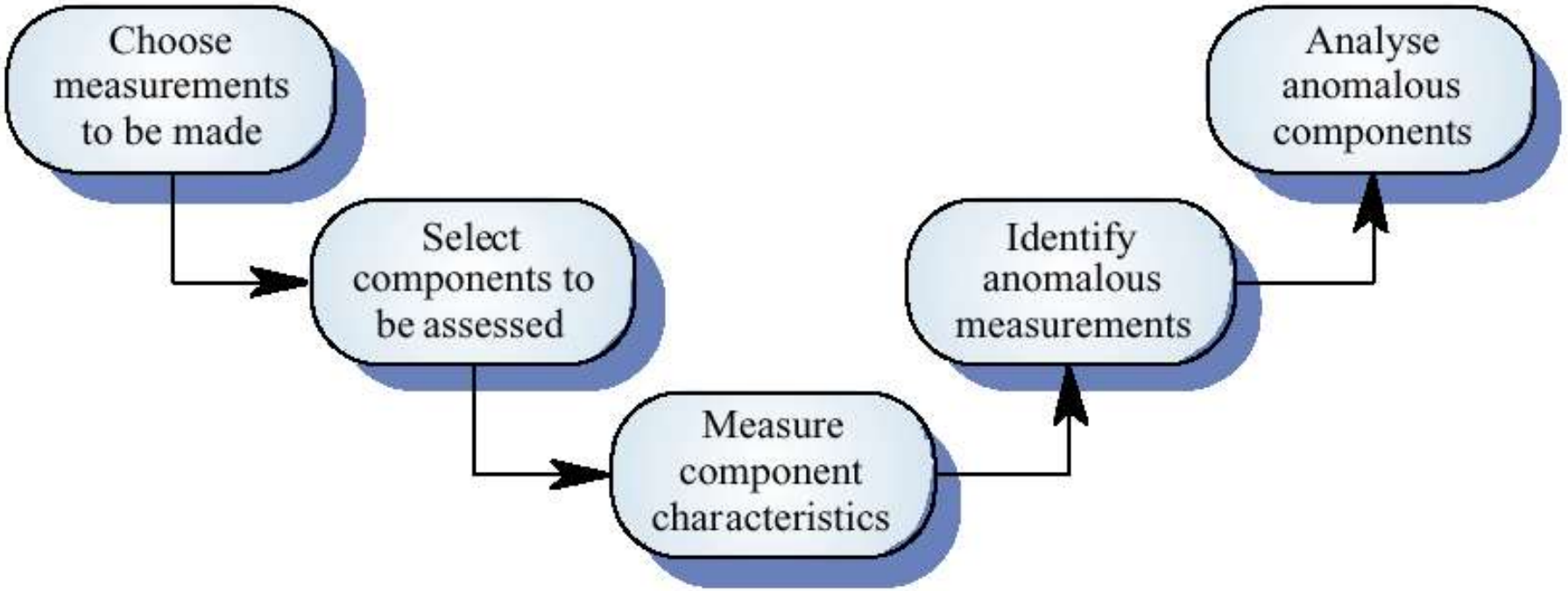
- Document the PMA results in a project experience report
  - Project description
  - Projects main problems, with description and Ishikawa diagrams
  - Project main success, with descriptions and Ishikawa diagrams
  - PMA meeting as an appendix (to let the reader see how the team discussed problems and successes)

Collecting and using measures

The measurement process

- A process should be defined and implemented to collect data, derive and analyze measures
- Data collected during this process should be maintained as an organisational resource
- Once a measurement database has been established, comparisons across projects become possible

Product measurement process



GQM

- Focus on few, important measures (top down)
- Never “collect everything, analyze later” (bottom up)
  - Too much data
  - Not meaningful

Goal - (similar to KPI)

- G1Satisfying customer
  - What is satisfaction?
    - Interviews
  - What is quality of product?
    - Defects after delivery
- G2 produce low cost product
  - What is cost
    - Cost of development

GQM example

- Overall research question
  - Are UML Object diagrams useful?

Goal

- Object of study
  - UML Static structure diagrams
- Purpose
  - Evaluate
- Focus
  - Usefulness
- Point of view
  - Maintainer comprehending software
- Context
  - Master degree class

Typical indicators

- Effort (Cost)
- Size
- Defects after delivery
- Defects during development

Data collection

- A metrics programme should be based on a set of product and process data
- Data should be collected immediately (not in retrospect) and, if possible, automatically
- Data should be controlled and validated as soon as possible

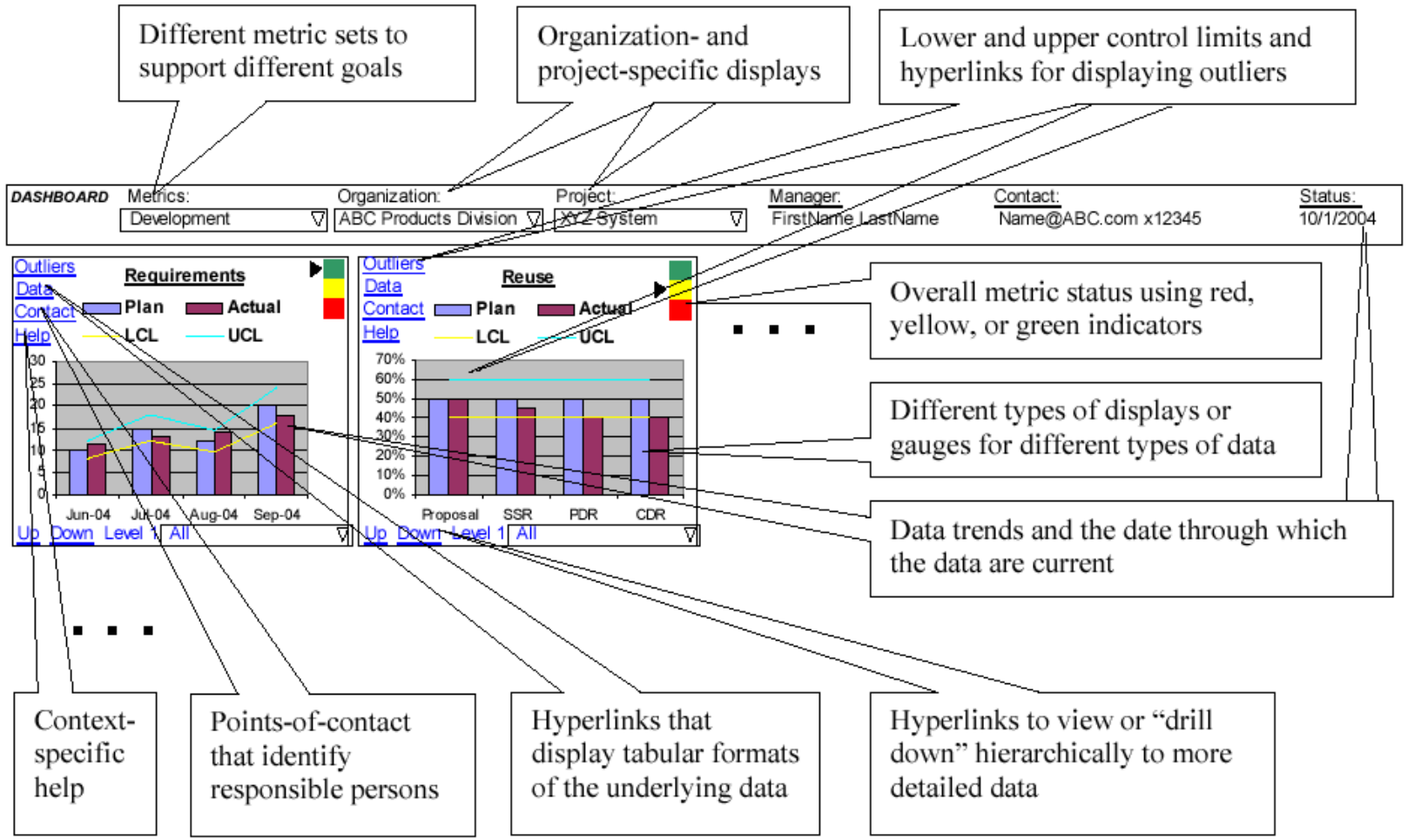
Data accuracy

- Don’t collect unnecessary data
  - The questions to be answered should be decided in advance and the required data identified
- Tell people why the data is being collected
  - It should not be part of personnel evaluation
- Don’t rely on memory
  - Collect data when it is generated not after a project has finished

Data presentation

- Reports
- Web reports
- Dashboard

Dashboard



Personnel

Project personnel

- Key activities requiring personnel:
  - requirements analysis
  - system design
  - software design
  - implementation
  - testing
  - training
  - maintenance
  - quality assurance

Choosing personnel

- ability to perform work
- interest in work
- experience with
  - similar applications
  - similar tools or languages
  - similar techniques
  - similar development environments
- training
- ability to communicate with others
- ability to share responsibility
- management skills

Work styles

- Extroverts: tell their thoughts
- Introverts: ask for suggestions
- Intuitives: base decisions on feelings
- Rationals: base decisions on facts, options

Organizational structure

- Depends on
  - backgrounds and work styles of team members
  - number of people on team
    - n people, max interactions = $n^2/2$
  - management styles of customers and developers
- Examples:
  - Chief programmer team
  - Egoless approach

Organizational structures

Highly structured

- high certainty
- repetition
- large project

Loosely structured

- uncertainty
- new technology
- small projects

Risk management

- Nothing is easy as it looks
- Everything takes longer than you expect
- And if anything can go wrong, it will, at the worst possible moment

[Murphy’s law]

Risk management

- Project Management for adults

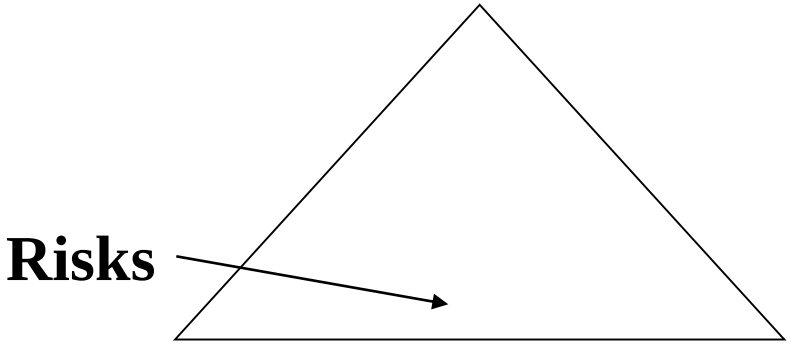
If you don’t actively attack the risks,

they will actively attack you

Tom Gilb

Risk Management

Software system (functions, quality)



Calendar time

Costs

Risk

- Future event that can have (bad) impact on project

Strategies

- Reactive
  - “Indiana Jones school of risk management”
  - Risk management = Crisis management (“fire-fighting mode”)
- Proactive

Risk management (proactive)

- Identify risks
- analyze them
- quantify effects
- define strategies and plans to handle them

Risk categories

- Project
- Technical
- Business

- Known
- Predictable
- Unknown

Project Risks

- Regarding (ill defined) project plan
  - budget, personnel, timings, resources, customers
- Regarding management
  - No management support
  - Missing budget or people

Technical risks

- Regard feasibility of product
  - Design, interfaces, verification, ..

Business risks

- Regarding market or company
  - No market for the product (*market risk*)
  - Product not in scope with company plans (*strategic risk*)
  - Sales force does not know how to sell the product (*sales risk*)

Known risks

- Identified analyzing project characteristics
- Ex:
  - Unrealistic deadlines
  - No requirements
  - No focus
  - Poor development environment

Predictable risks

- Identified analyzing previous projects
- Ex.
  - Personnel turnover
  - Poor communication with customer

Unknown

- Hard or impossible to identify..

Boehm’s top ten risk items

- Personnel shortfalls
- Unrealistic schedules and budgets
- Developing the wrong functions
- Developing the wrong user interfaces
- Gold-plating
- Continuing stream of requirements changes
- Shortfalls in externally-performed tasks
- Shortfalls in externally-furnished components
- Real-time performance shortfalls
- Straining computer science capabilities

Other common risks

- instability of COTS (Commercial Off-The-Shelf) components/products
- interface with legacy
- stability of development platform (hw + sw)
- limitations of platform
- multi-site development
- use of new methodologies / technologies
- standards, laws
- development team involved in other activities
- communication/language problems

Risk management terms

- Risk impact: the loss associated with the event
- Risk probability: the likelihood that the event will occur
- Risk control: the degree to which we can change the outcome

Risk exposure = (risk probability) x (risk impact)

RM Process

- 1- Risk assessment
  - identification
  - analysis
  - ranking
- 2- Risk control
  - planning
  - monitoring

Identification

- identify risks
  - checklist, taxonomies, questionnaires
    - PMI (Project Management Institute, PMBOK)
    - SEI (SEI-93-TR-06)
      - ex: technical, management, business risks
  - brainstorming
  - experience

Analysis

- probability
  - very high, high, medium, low, very low
- impact
  - catastrophic, critical, marginal, negligible
- exposure
  - probability * impact

Exposure

Impact/ probability	Very high	High	Medium	Low	Very low
Catastrophic	High	High	Moderate	Moderate	Low
Critical	High	High	Moderate	Low	Null
Marginal	Moderate	Moderate	Low	Null	Null
Negligible	Moderate	Low	Low	Null	Null

Ranking

- By exposure
- by qualitative assessments
  - only higher exposure risks are handled

RM Process

- 1- Risk assessment
  - identification
  - analysis
  - ranking
- 2- Risk control
  - planning
  - monitoring

Planning

- For selected risks (high in exposure)
  - define corrective actions
  - evaluate cost, decide if acceptable
  - insert actions in project plan

Three strategies for risk reduction

- avoiding the risk: change requirements for performance or functionality
- transferring the risk: transfer to other system, or buy insurance
- assuming the risk: accept and control it

risk leverage = difference in risk exposure divided by cost of reducing the risk

Ex.

- ABS for car, software controlled. More flexible, but risk of failure from software
  - Avoiding. No software controlled
  - Transfer. Insurance.
  - Assuming. Develop software with best techniques, apply redundancy.

Example

- Risk leverage
  - ABS, software developed normally
    - cost 100KEuro,
    - risk exposure = 10^{-3} * 1M Euro
  - ABS, software developed best techniques
    - cost 1M Euro,
    - risk exposure = 10^{-6} * 1M Euro
  - Risk leverage

    (10^{-3} * 1M Euro - 10^{-6} * 1M Euro) /
    (1M – 100k)Euro

Company profiles and risk handling styles

- project owner - takes charge of risk
- fixed price contract
- work provider - no interest in risk

Monitoring

- follow project plan, including corrective actions
- monitor status of risks
- identify new risks, assess them, update ranking

Monitoring (2)

- Is part of PM that has to consider also
  - risk log (document)
  - risk reviews (activities)
    - also with external assessors
    - can be coupled with project reviews

Risk log

Risk	Probability	Impact	Exposure	Action	Status
hw platform not available	high	Critical	high	Add software Layer compatible with other platforms	Under control

Actions for risks

- Personnel shortfalls
  - hire the best, the most suitable, training, team building, technical reviews
- unrealistic schedules and budget
  - more detailed plans, iterative process, reuse, new plans
- instability of components (COTS)
  - qualification, detailed analysis of product and vendor, software layer.

- inadequate requirements
  - prototyping, JAD, iterative process, include user representative in process
    - Joint Application Development
- inadequate user interface
  - study user needs, usability analysis, prototyping
- requirement changes
  - suitable design, iterative process, rigid change control

- Interface with legacy
  - reengineering, encapsulation, incremental change
- subcontractors
  - contracts and payments, team building, assessments before and during
- new technologies
  - prototyping, cost benefit analysis

References

- www.pmi.org project management institute
- www.sei.cmu.edu
- www.ifpug.org
- http://www.itmpi-journal.com
- www.fhg.iese.de – Fraunhofer IESE
- Rapid Development - Taming Wild Software Schedules, Steve McConnell, Microsoft Press, 1996
- Software Engineering Risk Management, Dale Walter Karolak, IEEE Computer Society Press, 1996
- Assessment and Control of Software Risks , Caper Jones, Yourdon Press, 1994
- Software Risk Management - Principles and Practices, Barry W.Boehm, IEEE Software, Vol 8, No. 1, Jan 1991, PP32-41
- Taxonomy-Based Risk Identification, M.J.Carr et al., CMU/SEI-93-TR-06, SEI, 1993
- Www.riskwatch.com - Risk management tools